

Pontifícia Universidade Católica de São Paulo
PUC-SP



PUC-SP

Vinicius de Lucena

Sistema de Gestão para ONG de Adoção de Animais: Modelagem e Implementação
de um Banco de Dados Relacional com SQLite e Streamlit

Ciência de Dados e Inteligência Artificial

São Paulo
2026

SUMÁRIO

1	INTRODUÇÃO	2
2	FUNDAMENTAÇÃO TEÓRICA	3
2.1	MODELO RELACIONAL E O SGBD SQLITE	3
3	METODOLOGIA	5
3.1	DIAGNÓSTICO INICIAL	5
3.5	INSERÇÃO DE DADOS DE TESTE	7
4	RESULTADOS E DISCUSSÃO	7
4.1	ENTREGÁVEIS E VALIDAÇÃO	7
4.2	TRATAMENTO DE CONCORRÊNCIA E INTEGRIDADE.....	7
5	APLICAÇÃO NO STREAMLIT.....	8
6	CONCLUSÕES	8
	REFERÊNCIAS	9

1 INTRODUÇÃO

O trabalho desenvolvido aborda o problema da organização e recuperação de informações de uma ONG de adoção de animais localizada em São José dos Campos (SP), que realiza adoções de cães e gatos e oferece serviço de castração para a comunidade. O contato com a instituição foi estabelecido como parte deste projeto acadêmico, com o objetivo de compreender suas dores e necessidades quanto ao armazenamento de dados.

O objetivo é implementar um sistema completo de banco de dados (diagnóstico, modelagem, implementação em um SGBD, inserção de dados de teste e documentação), acompanhado de uma interface simples de uso, capaz de substituir o processo manual atual da instituição.

Atualmente, a ONG coleta informações por meio de três questionários distintos no Google Forms (adoção de cães, adoção de gatos e castração), cujas respostas são exportadas manualmente para planilhas Excel sem integração entre si. Esse processo gera fragmentação dos dados, ausência de controle de status das solicitações e dificuldade de rastrear o histórico de adotantes e animais. O sistema proposto centraliza essas informações em um único banco de dados relacional, com controle de acesso e fluxos de aprovação.

2 FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os conceitos que fundamentam as decisões técnicas do trabalho.

3 MODELO RELACIONAL E O SGBD SQLITE

O modelo relacional organiza os dados em tabelas (relações) compostas por linhas e colunas, nas quais os relacionamentos entre entidades são expressos por chaves estrangeiras. Essa abordagem é adequada ao domínio da ONG, em que as informações são fortemente relacionadas: um adotante faz solicitações, uma solicitação refere-se a um animal, e um animal pode receber várias solicitações.

O SGBD escolhido foi o SQLite, um banco de dados relacional que armazena toda a base em um único arquivo e dispensa um servidor dedicado. Diferentemente de bancos cliente-servidor, o SQLite é embarcado: a própria aplicação acessa o arquivo diretamente. Essa característica o torna apropriado para uma instituição de pequeno porte, sem equipe de tecnologia, na qual a simplicidade de instalação e de backup é mais valiosa do que a escalabilidade de um servidor.

3.1 INTEGRIDADE DE DADOS E RESTRIÇÕES

A integridade dos dados é garantida por restrições aplicadas pelo próprio SGBD. Foram utilizadas chaves primárias (identificação única de cada registro), chaves estrangeiras (integridade referencial entre tabelas), restrições de unicidade (em campos como e-mail e CPF) e restrições de domínio do tipo CHECK, que limitam os valores aceitos em colunas de status, espécie e sexo.

Um recurso central do projeto é o índice único parcial, que permite impor unicidade apenas a um subconjunto das linhas. Ele foi empregado para garantir que cada animal tenha, no máximo, uma adoção com status aprovado, uma regra de negócio crítica que assim passa a ser garantida pelo banco, e não apenas pela aplicação.

3.2 NORMALIZAÇÃO

A normalização é o processo de organizar as tabelas para reduzir redundância e evitar anomalias de atualização. O modelo foi normalizado até a Terceira Forma Normal (3FN): os atributos são atômicos (1FN), não há dependências parciais de chave (2FN) e nenhum atributo não-chave depende transitivamente de outro (3FN). Por exemplo, os dados pessoais do adotante foram separados da conta de usuário, e

os dados do animal não são duplicados nas solicitações, sendo referenciados por chave estrangeira.

4 METODOLOGIA

Descrição do diagnóstico, da modelagem, do SGBD e arquitetura, da implementação e dos dados de teste.

5 DIAGNÓSTICO INICIAL

A partir das informações levantadas junto à instituição, identifiquei as seguintes dores: processo manual suscetível a erros, dados fragmentados em planilhas sem relação, ausência de controle de status das solicitações, dificuldade de rastreamento do histórico e ausência de um controle centralizado de animais disponíveis versus adotados. Essas dores justificam a necessidade de um banco de dados que centralize e relacione as informações.

5.1 MODELAGEM DO BANCO DE DADOS

O modelo é composto por seis entidades: usuario (conta de acesso), dono (perfil estendido do adotante, em relação 1:1 com usuário), animal, solicitacao_adocao, solicitacao_castracao e log_auditoria. Os principais relacionamentos são: um usuário pode possuir um perfil de dono; um dono pode fazer várias solicitações de adoção; um animal pode receber várias solicitações (mas apenas uma pode ser aprovada); e um usuário pode registrar várias solicitações de castração.

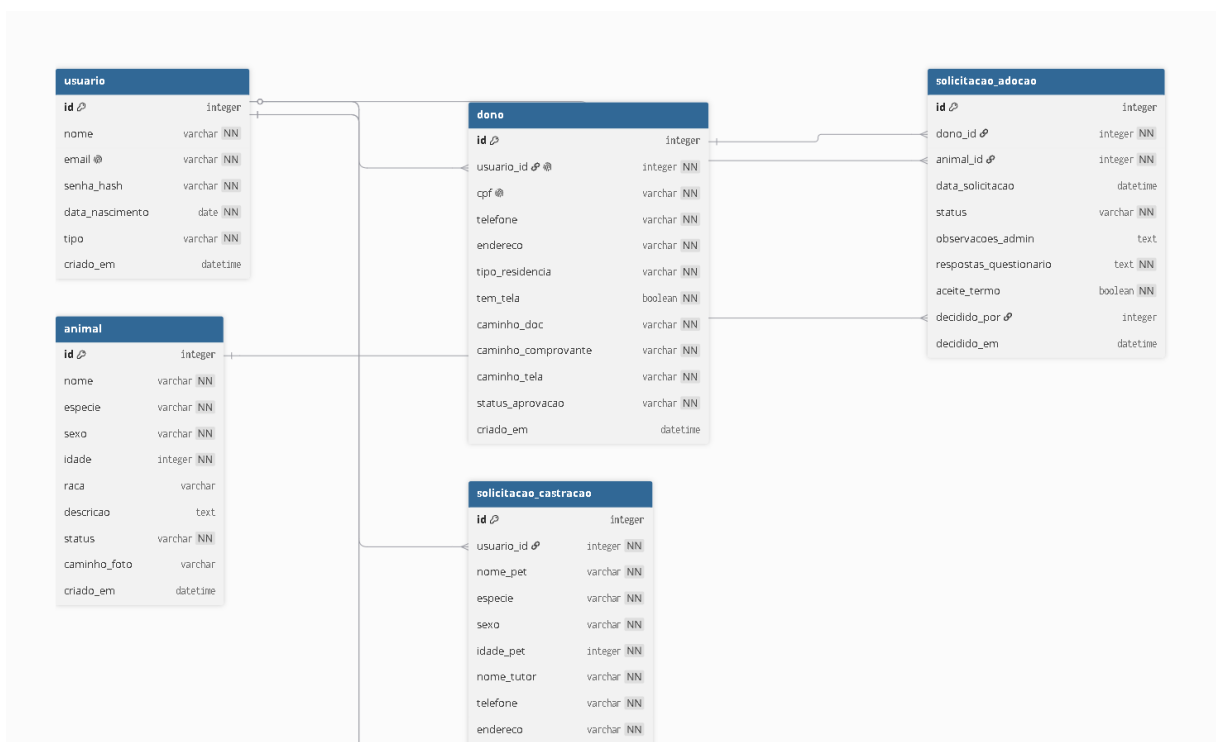


Figura 1 - Diagrama Entidade-Relacionamento, gerado no dbdiagram.io a partir de docs/diagrama_er.dbml

5.2 SGBD ESCOLHIDO E ARQUITETURA DA APLICAÇÃO

Como discutido na fundamentação, optei pelo SQLite por sua adequação ao porte da instituição. A aplicação foi desenvolvida em Python com a biblioteca Streamlit para a interface, e segue uma arquitetura em camadas com dependências unidirecionais: apresentação (telas Streamlit), serviços (regras de negócio), repositórios (acesso a dados) e o SGBD. Essa separação facilita a manutenção e os testes, pois cada camada possui uma responsabilidade única e não conhece as camadas superiores.

5.3 IMPLEMENTAÇÃO DAS REGRAS DE NEGÓCIO

As regras da ONG foram traduzidas em código e em restrições do banco. O acesso é controlado por perfis (RBAC): um código de acesso fornecido pela ONG, informado no cadastro, define se a conta é de administrador ou de usuário comum. As senhas são armazenadas exclusivamente com bcrypt, nunca em texto puro, e todas as consultas utilizam comandos parametrizados para prevenir injeção de SQL.

A aprovação de uma adoção é a operação mais sensível e foi implementada de forma transacional: em uma única transação, a solicitação é aprovada, o animal é marcado como adotado, as demais solicitações em análise para aquele animal são

reprovadas e o aceite do termo de compromisso é registrado. Caso qualquer etapa falhe, a transação é revertida (rollback), preservando a consistência.

6 INSERÇÃO DE DADOS DE TESTE

Para validação, populei o banco com dados fictícios: um administrador e três usuários comuns em diferentes etapas do processo; três cães e três gatos disponíveis e dois animais adotados; cinco solicitações de adoção (duas em análise, duas aprovadas e uma reprovada); e duas solicitações de castração. Vale registrar uma decisão de consistência: como cada animal adotado deve possuir exatamente uma adoção aprovada, utilizei duas adoções aprovadas (uma para cada animal adotado), ajustando o enunciado original para preservar a integridade do modelo.



solicitacao_id	data_solicitacao	decidido_em	usuario_id	nome_adotante	email_adotante	cpf	telefone	endereco	animal_id	nome_animal	especie	status
1	2026-06-19 19:11:32	2026-06-19 19:11:32	2	Maria Silva	maria@email.com	12345678901	(11) 90000-0001	Rua das Flores, 100 - Sao Paulo/SP	7	Bob	cao	ma
2	2026-06-19 19:11:32	2026-06-19 19:11:32	4	Ana Souza	ana@email.com	34567890123	(11) 90000-0003	Rua Verde, 75 - Sao Paulo/SP	8	Lola	gato	fe

Figura 2 - Dados de teste visualizados no DB Browser for SQLite

7 RESULTADOS E DISCUSSÃO

8 ENTREGÁVEIS E VALIDAÇÃO

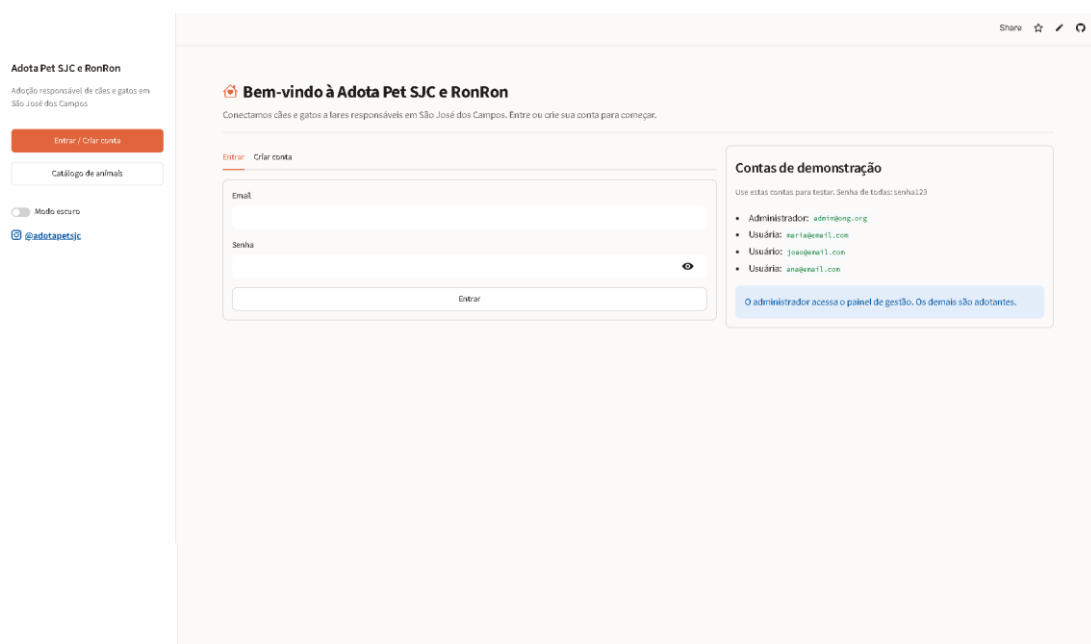
O resultado é um banco de dados relacional funcional, acompanhado de uma aplicação web que cobre os fluxos de cadastro, autenticação, complemento de perfil de adotante, catálogo de animais, solicitação e aprovação de adoção, solicitação e agendamento de castração, e um painel administrativo com auditoria manual de documentos. A validação foi feita com os dados de teste e com uma consulta que verifica a invariante de consistência: todo animal adotado possui exatamente uma adoção aprovada.

9 TRATAMENTO DE CONCORRÊNCIA E INTEGRIDADE

Mesmo em um cenário com poucos operadores, o sistema foi projetado para não permitir estados inconsistentes. Caso dois administradores tentem aprovar adoções para o mesmo animal, a primeira operação é concluída e a segunda viola o índice único parcial, levantando um erro de integridade que é convertido em uma mensagem de negócio clara (animal já adotado). Assim, a integridade é garantida pelo banco como última linha de defesa, e não apenas pela lógica da aplicação.

10 APLICAÇÃO NO STREAMLIT

Para melhor visibilidade e teste das funcionalidades, desenvolvi uma aplicação com interface visual em Streamlit. A navegação é montada dinamicamente conforme o perfil do usuário autenticado. O usuário comum acessa o catálogo de animais disponíveis, o complemento do seu perfil de adotante, a solicitação de adoção (com questionário de entrevista) e a solicitação de castração. O administrador acessa, adicionalmente, um painel com a verificação manual dos documentos enviados, o CRUD de animais, a decisão das solicitações de adoção, a agenda de castração e o log de auditoria.



Figuras 3, 4 e 5 - Imagens das telas da Aplicação Streamlit

(<https://adotapetpucsp.streamlit.app>)

11 CONCLUSÕES

O projeto, guiado pelos conteúdos da disciplina, resultou em uma solução de banco de dados consistente e adequada ao problema da instituição. A escolha do SQLite foi orientada pelo contexto da ONG (porte pequeno, ausência de equipe técnica e necessidade de simplicidade), e não pela ferramenta mais complexa disponível. Ainda assim, recursos como integridade referencial, restrições de domínio, índice único parcial e transações permitiram garantir a robustez do negócio.

Cabe destacar que a modelagem é portátil: caso a operação cresça e exija acesso remoto, a migração para um SGBD cliente-servidor pode ser feita sem alterar o desenho das tabelas. As ferramentas de IA generativa foram utilizadas como apoio na construção da aplicação e na revisão pontual de código e texto; a modelagem, as decisões técnicas e a análise dos resultados foram de minha autoria, com base nos conteúdos abordados em sala de aula.

REFERÊNCIAS

SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: jun. 2026.

SQLite, Partial Indexes. Disponível em: <https://www.sqlite.org/partialindex.html>. Acesso em: jun. 2026.

Streamlit Documentation. Disponível em: <https://docs.streamlit.io/>. Acesso em: jun. 2026.